

Peso 1

1. Com base no material, qual é a diferença fundamental entre Git e GitHub?

- A) Git é a ferramenta local de controle de versão; GitHub hospeda repositórios Git e recursos colaborativos.
- B) Git é uma rede social para programadores; GitHub é o sistema de arquivos local.
- C) Git e GitHub são a mesma ferramenta, diferenciando-se apenas pelo sistema operacional.
- D) Git é um servidor autoritativo centralizado; GitHub é o sistema distribuído local.
- E) Git é usado apenas para operações de rede; GitHub armazena os metadados de hash SHA-1.

Peso 2

2. Ao tentar publicar commits locais no GitHub usando `git push`, a operação é rejeitada. Segundo os slides, qual é a principal causa técnica dessa rejeição?

- A) O repositório local está usando um modelo centralizado, incompatível com o GitHub.
- B) O arquivo `.gitignore` bloqueou a transferência de arquivos pesados.
- C) O repositório remoto contém commits que a sua máquina local ainda não possui.
- D) A branch local foi configurada para ignorar atualizações da Staging Area.
- E) O comando `git add` não foi executado no repositório remoto.

Peso 3

3. Você realizou diversas alterações, usou `git add` e criou um commit local com uma mensagem incorreta ("teste"). O commit ainda não foi enviado ao GitHub. De acordo com os princípios de recuperação de erros do material, qual é a ação mais adequada para desfazer o commit mantendo as alterações preparadas na Staging Area para reescrever a mensagem?

- A) Executar `git revert <hash>`.
- B) Executar `git reset --hard HEAD~1`.
- C) Executar `git restore --staged <arquivo>`.
- D) Executar `git reset --soft HEAD~1`.
- E) Executar `git reset --mixed HEAD~1`.

Peso 1

4. No modelo de dados interno do Git, o que o objeto do tipo "Blob" armazena?

- A) A estrutura de diretórios, incluindo nomes e permissões.
- B) O conteúdo de um arquivo, sem nome ou caminho.
- C) Um snapshot histórico com metadados de autoria.
- D) Uma referência nomeada para uma versão estável.
- E) A configuração global do usuário, como nome e e-mail.

Peso 2

5. Você acabou de criar um novo arquivo no seu diretório de trabalho, mas o Git ainda não o acompanha. Em qual estado este arquivo se encontra?

- A) Modified
- B) Staged
- C) Committed
- D) Untracked
- E) Tracked

Peso 3

6. Um colega fez alterações no projeto hospedado no GitHub. Você deseja verificar se há mudanças e comparar o histórico remoto com o local, mas sem alterar os arquivos do seu diretório de trabalho local neste momento. Qual comando você deve utilizar?

- A) `git pull origin main`
- B) `git push origin main`
- C) `git fetch origin`
- D) `git clone`
- E) `git log --oneline`

Peso 1

7. A evolução dos sistemas de controle de versão passou por arquiteturas locais, centralizadas e distribuídas. Qual é a principal característica da arquitetura distribuída, adotada pelo Git?

- A) Dependência de um servidor único que armazena o histórico autoritativo.
- B) O histórico é armazenado na máquina do usuário, mas não suporta colaboração.

- C) Cada clone contém uma cópia completa do histórico do projeto.
- D) Os arquivos são armazenados apenas como listas de linhas adicionadas e removidas.
- E) A impossibilidade de registrar progresso local sem conexão à internet.

Peso 2

8. O modelo das três árvores é central para o funcionamento do Git. Qual é o papel específico da camada Staging Area (Index)?

- A) Funcionar como a camada visível onde os arquivos são editados manualmente.
- B) Atuar como área de curadoria e seleção da versão dos arquivos para o próximo snapshot.
- C) Armazenar o histórico persistente de commits criados no repositório.
- D) Configurar a identidade do autor do projeto e suas credenciais.
- E) Manter o backup automático de todas as alterações salvas no computador.

Peso 3

9. Maria tentou executar um `git push` para enviar seus commits locais, mas a operação foi rejeitada pelo GitHub porque o histórico remoto avançou. Baseando-se exclusivamente no procedimento de diagnóstico recomendado nos slides, qual é a sequência correta e segura de comandos para resolver a situação e publicar seu trabalho?

- A) `git push --force -> git status -> git pull`
- B) `git status -> git fetch origin -> git pull origin main -> git push`
- C) `git log -> git reset --hard -> git push`
- D) `git pull origin main -> git fetch origin -> git push --force`
- E) `git fetch origin -> git push -> git status`

Peso 1

10. Qual é a finalidade principal do arquivo `.gitignore` apresentado no material?

- A) Declarar padrões de arquivos que o Git deve ignorar quando ainda não estão rastreados, evitando versionar arquivos temporários ou sensíveis.
- B) Apagar automaticamente do histórico todos os commits que contenham erros lógicos.
- C) Ignorar as regras de configuração globais do sistema operacional.
- D) Forçar a sincronização de arquivos binários pesados com o repositório remoto.

E) Remover arquivos sensíveis que já foram enviados e compartilhados no GitHub.

Peso 2

11. Os comandos de inspeção fornecem informações em diferentes níveis. Enquanto o `git status` indica que um arquivo foi alterado, qual comando permite observar exatamente o que mudou, linha a linha?

- A) `git log`
- B) `git init`
- C) `git remote -v`
- D) `git diff`
- E) `git restore`

Peso 3

12. Um estudante corrigiu um erro no cálculo de divisão de um script e deseja realizar um commit atômico usando a padronização Conventional Commits explicada no material. Qual das opções abaixo representa o padrão e o tipo corretos para esta situação?

- A) `docs: altera calculo da divisao`
- B) `feat: script novo criado`
- C) `fix: corrige divisao por zero`
- D) `chore: divisao consertada`
- E) `update: corrige erro no script`

Peso 1

13. O Git identifica objetos através de um hash SHA-1 de 40 caracteres. O que acontece com esse hash se um único caractere ou espaço do arquivo for alterado?

- A) O hash é encurtado para economizar espaço de armazenamento.
- B) O hash muda completamente, gerando um novo identificador.
- C) O hash permanece o mesmo para garantir a continuidade do histórico.
- D) O Git atualiza apenas os 4 últimos caracteres do hash.
- E) O objeto original é sobrescrito, mantendo o mesmo identificador no banco de dados.

Peso 2

14. Dentro da pasta oculta `.git`, existe um componente chamado `HEAD`. Qual é a sua função, segundo os slides?

- A) Armazenar as credenciais e chaves SSH do usuário.
- B) Configurar as quebras de linha entre diferentes sistemas operacionais.
- C) Guardar todos os arquivos físicos do diretório de trabalho.
- D) Atuar como um ponteiro/referência para o estado (branch ou commit) atualmente ativo.
- E) Representar o banco de dados principal de objetos (blobs, trees, tags).

Peso 3

15. Um desenvolvedor iniciante commitou e enviou (via push) o arquivo `.env`, contendo a senha do banco de dados, para um repositório público no GitHub. De acordo com as diretrizes de higiene e segurança do material, qual deve ser a primeira e principal atitude do desenvolvedor em relação a esse vazamento?

- A) Utilizar `git restore --staged .env` para remover o arquivo do GitHub.
- B) Incluir o arquivo no `.gitignore` e tentar um `git push` novamente.
- C) Executar `git reset --hard` no repositório local.
- D) A senha deve ser considerada comprometida e a credencial deve ser imediatamente revogada no serviço correspondente.
- E) Excluir a pasta `.git` do seu computador local e clonar novamente.

Peso 1

16. A configuração `core.autocrlf` resolve um problema comum em equipes com diferentes sistemas operacionais. A que ela se refere?

- A) Ao tratamento de hashes criptográficos.
- B) À padronização da identidade global (nome e e-mail).
- C) Ao tratamento de quebras de linha (CRLF no Windows e LF no Linux/macOS).
- D) À definição do editor de texto padrão.
- E) À configuração da autenticação via chaves SSH.

Peso 2

17. O material destaca que o Git não funciona guardando apenas listas de linhas adicionadas e removidas (deltas) a cada commit. Como o Git registra de fato os estados do projeto?

- A) Capturando um snapshot (instantâneo lógico) completo dos arquivos naquele momento, reutilizando referências para objetos que não mudaram.
- B) Duplicando fisicamente todos os arquivos do projeto a cada novo commit.
- C) Armazenando as diferenças linha a linha em um servidor central remoto.
- D) Mantendo um log temporário que é deletado quando o comando push é executado.
- E) Compactando a pasta inteira em um arquivo ZIP dentro do diretório refs/.

Peso 3

18. Durante seu trabalho, você modificou os arquivos `index.html` e `teste.js`. Você executou `git add teste.js` por engano, pois desejava que essa alteração ficasse fora do próximo commit atômico. Como você pode remover o arquivo da Staging Area sem perder o conteúdo modificado no seu diretório de trabalho?

- A) `git reset --hard HEAD~1`
- B) `git restore --staged teste.js`
- C) `git restore teste.js`
- D) `git revert <hash>`
- E) `git rm --cached teste.js`

Peso 1

19. Qual comando permite visualizar a lista de repositórios remotos configurados e suas respectivas URLs de fetch e push?

- A) `git status`
- B) `git remote add origin`
- C) `git log --oneline`
- D) `git remote -v`
- E) `git diff`

Peso 2

20. Segundo a padronização Conventional Commits, qual termo é recomendado para indicar uma nova funcionalidade ou comportamento observável no sistema?

- A) `fix`
- B) `feat`

C) refactor

D) chore

E) perf

Peso 3

21. Você editou três arquivos no seu projeto: atualizou a documentação no `README.md`, corrigiu um erro de lógica em `calcula.py`, e adicionou anotações aleatórias em `rascunho.txt`. Considerando a prática de commits atômicos do módulo, qual é a forma correta de registrar essas alterações?

A) Executar `git add .` e criar um commit único com a mensagem "atualiza coisas".

B) Adicionar todos os três arquivos e commitar como "fix: corrige calculo e atualiza readme".

C) Ignorar todos com um `.gitignore` e não fazer commit.

D) Executar comandos `git add` separados para `README.md` e `calcula.py`, criando dois commits distintos com mensagens claras, e não commitar o arquivo de rascunho.

E) Fazer o commit primeiro apenas do `rascunho.txt` e apagar o resto do diretório de trabalho.

Peso 1

22. O que o comando `git pull` executa por baixo dos panos na sua configuração padrão?

A) Envia as alterações locais diretamente para a nuvem.

B) Apaga o histórico local e substitui pelo remoto.

C) Exclui a branch remota.

D) Realiza a busca (fetch) das alterações remotas e as integra (merge) à branch local.

E) Inicia a configuração global do repositório.

Peso 2

23. O histórico do Git é estruturado como um Grafo Acíclico Direcionado (DAG). O que significa dizer que o histórico tem essa estrutura?

A) Significa que o Git empacota todas as versões antigas em formato linear irreversível.

B) Significa que a evolução ocorre localmente apenas na branch main.

C) Significa que cada commit aponta para seu commit pai, formando uma cadeia histórica direcional que nunca volta ciclicamente ao ponto de partida.

D) Significa que o histórico deve obrigatoriamente estar hospedado em um servidor distribuído como o GitHub.

E) Significa que branches não podem ser usadas para trabalhos paralelos.

Peso 3

24. Joana deseja iniciar um repositório local do zero, verificar o estado do repositório vazio, preparar um novo arquivo chamado `projeto.md` e, por fim, registrá-lo. Qual sequência de comandos reflete exatamente este fluxo de trabalho?

A) `git init -> git diff -> git commit -m "add projeto" -> git status`

B) `git init -> git status -> git add projeto.md -> git commit -m "docs: inicializa projeto"`

C) `git clone -> git log -> git pull -> git add projeto.md`

D) `git status -> git init -> git commit -m "add" -> git push`

E) `git add projeto.md -> git init -> git push -> git commit -m "docs: inicializa"`

Peso 1

25. Qual comando é indicado para desfazer, com segurança, os efeitos de um commit que já foi enviado ao GitHub e compartilhado com a equipe, criando um novo commit inverso?

A) `git revert`

B) `git reset --hard`

C) `git restore`

D) `git init`

E) `git fetch`

Peso 2

26. Sabendo que o objeto Blob armazena apenas conteúdo, qual objeto interno é responsável por mapear os nomes, permissões e caminhos para os Blobs e para si mesmo?

A) Tag

B) Commit

C) Tree

D) Index

E) Head

Peso 3

27. Um estudante estava trabalhando no arquivo `estilo.css` e realizou diversas alterações que acabaram quebrando o layout da página. Ele ainda não usou `git add` no arquivo. Baseado nas estratégias de recuperação ensinadas, qual comando ele deve utilizar para descartar totalmente suas mudanças locais e voltar ao estado salvo no último commit?

- A) `git reset --soft HEAD~1`
- B) `git restore estilo.css`
- C) `git revert HEAD`
- D) `git restore --staged estilo.css`
- E) `git pull origin main`

Peso 1

28. Ao consultar a documentação técnica no terminal através do comando `git help`, o que a convenção de colchetes `[ ]` indica na sintaxe de um comando?

- A) Variáveis obrigatórias que devem ser substituídas.
- B) Repetição ou lista infinita de opções.
- C) Fim do arquivo de ajuda (saída).
- D) Alternativas mutuamente excludentes.
- E) Argumentos opcionais.

Peso 2

29. Dentre as boas práticas e ética de versionamento citadas no material, o que é recomendado a respeito do uso da branch principal (`main` ou `master`) em projetos colaborativos em equipe?

- A) O estudante deve commitar diretamente nela para poupar tempo.
- B) Não se deve commitar diretamente na branch principal, pois ela deve representar uma linha estável do projeto.
- C) A branch principal deve ser evitada e excluída logo após a criação do repositório.
- D) Deve-se usar a branch principal apenas para armazenar backups do diretório `.git`.
- E) Commits gigantes e fragmentados são ideais para a branch principal.

Peso 3

30. Ao executar `git status`, Pedro notou que o arquivo `index.html` aparece listado duas vezes: uma vez como `staged` e outra vez como `modified`. Considerando o modelo de três árvores dos slides, o que causa essa situação?

- A) Um erro de configuração no arquivo `.gitignore`.
- B) A ocorrência de um conflito irreversível entre o repositório local e o GitHub, forçando a duplicação.
- C) O arquivo foi preparado com `git add` (indo para a Staging Area) e, em seguida, editado novamente no Working Directory antes da execução do commit.
- D) O Git copiou o arquivo porque detectou que ele possui dados sensíveis.
- E) O comando `git restore --staged` foi executado pela metade.

GABARITO

Questão	Resposta	Peso
1	A	1
2	C	2
3	D	3
4	B	1
5	D	2
6	C	3
7	C	1

8	B	2
9	B	3
10	A	1
11	D	2
12	C	3
13	B	1
14	D	2
15	D	3
16	C	1
17	A	2
18	B	3
19	D	1
20	B	2

21	D	3
22	D	1
23	C	2
24	B	3
25	A	1
26	C	2
27	B	3
28	E	1
29	B	2
30	C	3

EXPLICAÇÃO DAS RESPOSTAS

Questão 1 — Alternativa A A alternativa A está correta porque os slides estabelecem explicitamente que o Git é uma ferramenta instalada localmente, enquanto o GitHub é um serviço online que hospeda repositórios Git e adiciona recursos colaborativos. As outras estão erradas porque Git não é uma rede social, nem um servidor centralizado, e ambos não são a mesma ferramenta.

Questão 2 — Alternativa C A alternativa C está correta porque o material ensina que a rejeição de push ocorre quando "o repositório remoto contém commits que a sua máquina local ainda não possui". O Git bloqueia para proteger o histórico remoto. As outras estão erradas pois o erro não decorre de incompatibilidade de modelo centralizado, bloqueios do `.gitignore` ou ausência de `git add` no remoto.

Questão 3 — Alternativa D A alternativa D está correta porque o comando `git reset --soft HEAD~1` desfaz o commit mantendo as alterações preparadas na Staging Area. As outras estão erradas porque `revert` cria um novo commit e é usado para erros já publicados, `reset --hard` seria destrutivo, `restore --staged` apenas remove da Staging Area sem desfazer o commit, e `reset --mixed` limparia a Staging Area exigindo novo `git add`.

Questão 4 — Alternativa B A alternativa B está correta porque o objeto Blob armazena exclusivamente o conteúdo de um arquivo, sem nome ou caminho associado. As outras estão erradas pois nomes e diretórios são responsabilidade da Tree, o snapshot histórico é o Commit, a referência nomeada é a Tag, e configurações de usuário ficam no arquivo `config`.

Questão 5 — Alternativa D A alternativa D está correta porque `Untracked` descreve um arquivo que existe no diretório de trabalho, mas que o Git ainda não acompanha. As outras estão erradas porque `Modified` refere-se a um arquivo já conhecido que foi editado, `Staged` indica que foi preparado para commit, e `Committed` significa que já está armazenado no repositório.

Questão 6 — Alternativa C A alternativa C está correta porque `git fetch origin` atualiza as referências remotas e baixa informações do repositório sem alterar o diretório de trabalho local. As outras estão erradas porque `git pull` alteraria os arquivos locais, `git push` envia alterações locais em vez de baixá-las, e `git clone` copia um repositório inteiro inicialmente em vez de inspecionar atualizações de um existente.

Questão 7 — Alternativa C A alternativa C está correta porque a essência da arquitetura distribuída, presente nos slides, é que ao clonar um repositório, cada cópia contém o histórico completo, permitindo redundância e trabalho offline. As outras estão erradas porque depender de servidor único descreve sistemas centralizados e sistemas locais puros não resolvem colaboração.

Questão 8 — Alternativa B A alternativa B está correta porque a Staging Area (ou Index) é definida como a camada de curadoria, onde o usuário seleciona e prepara a versão do arquivo para o próximo commit. As outras estão erradas pois a camada de edição é o Working Directory, e o armazenamento de histórico é o Repository.

Questão 9 — Alternativa B A alternativa B está correta porque o fluxo ensinado para lidar com rejeição é iniciar o diagnóstico com `git status`, depois baixar referências com `git fetch origin`, integrar as alterações remotas com `git pull origin main` (ou realizar merge) e só então enviar o trabalho com `git push`. As outras estão erradas porque o material orienta fortemente a evitar o `git push --force` e a ordem lógica exige baixar e integrar (fetch e pull) antes de enviar.

Questão 10 — Alternativa A A alternativa A está correta porque a função do `.gitignore` explicada no material é declarar padrões de arquivos que devem ser ignorados pelo controle de versão para não rastrear produtos derivados ou arquivos sensíveis. As outras estão erradas porque ele não apaga commits lógicos, não ignora configurações de sistema, não lida com sincronização de binários e não apaga retroativamente senhas enviadas ao GitHub.

Questão 11 — Alternativa D A alternativa D está correta porque o comando `git diff` é explicitamente apresentado como aquele que mostra o conteúdo da mudança linha a linha. As outras estão erradas porque `status` apenas orienta sobre o estado, `log` exibe o histórico de commits, e `restore` descarta mudanças locais.

Questão 12 — Alternativa C A alternativa C está correta porque `fix` é a palavra-chave utilizada no padrão Conventional Commits para denotar correção de erros, e a mensagem descreve o problema de forma objetiva ("corrige divisao por zero"). As outras estão erradas porque `docs` serve para documentação, `feat` indica funcionalidade nova, `chore` serve para tarefas auxiliares e configuração, e `update` não faz parte das categorias listadas nos slides.

Questão 13 — Alternativa B A alternativa B está correta porque qualquer alteração mínima (como um caractere ou um espaço) resulta em um hash SHA-1 completamente diferente, sustentando a integridade do histórico. As outras estão erradas porque o Git não encurta ou atualiza partes isoladas do hash, e hashes nunca permanecem iguais diante de modificações no arquivo.

Questão 14 — Alternativa D A alternativa D está correta pois o `HEAD` é explicitamente descrito como o ponteiro para o estado atualmente ativo (a branch ou o commit atual). As outras estão erradas porque credenciais e chaves não ficam ali, quebras de linha são em `config`, e o banco de dados principal de objetos é a pasta `objects/`.

Questão 15 — Alternativa D A alternativa D está correta porque o slide reforça a ética do versionamento e resposta a incidentes: se uma credencial for enviada ao GitHub, ela deve ser considerada comprometida e sua imediata revogação é prioridade, pois agentes automatizados rastreiam chaves públicas. As outras estão erradas porque comandos locais (como `restore` ou apagar o `.git`) não removeriam a senha do servidor e do acesso de terceiros no GitHub.

Questão 16 — Alternativa C A alternativa C está correta pois a configuração `core.autocrlf` resolve a diferença do tratamento de quebras de linha entre ambientes Windows (CRLF) e sistemas Unix como macOS ou Linux (LF). As outras opções listam funcionalidades que não têm ligação com a variável `autocrlf` e suas consequências no histórico.

Questão 17 — Alternativa A A alternativa A está correta porque o Git baseia seu modelo de dados em estados completos lógicos do projeto ("snapshots") e reutiliza as referências aos objetos previamente armazenados para garantir eficiência caso o arquivo não tenha mudado. As outras estão erradas porque o Git não duplica arquivos desnecessariamente a cada commit, e não funciona puramente com armazenagem de deltas (diferenças).

Questão 18 — Alternativa B A alternativa B está correta porque o comando `git restore --staged <arquivo>` permite remover um arquivo da Staging Area sem descartar ou apagar as modificações realizadas no diretório de trabalho. As outras estão erradas porque `restore` apagaria as modificações locais do diretório de trabalho, e `reset --hard` faria descarte destrutivo. O `rm --cached` é voltado à remoção do índice de arquivos já commitados ou em remoção completa de rastreo, diferente do fluxo básico de staging.

Questão 19 — Alternativa D A alternativa D está correta pois `git remote -v` exibe as URLs configuradas para fetch e push associadas aos repositórios remotos. As outras opções

estão erradas, já que `git remote add` adiciona um remote, e os demais (status, log, diff) operam nas árvores e histórico, não nos links de repositório.

Questão 20 — Alternativa B A alternativa B está correta porque `feat` é a indicação para "Nova funcionalidade ou comportamento observável" na tabela de Conventional Commits fornecida. As outras opções possuem finalidades distintas (`fix` para erro, `chore` para tarefas auxiliares, `refactor` para reestruturação, `perf` para desempenho).

Questão 21 — Alternativa D A alternativa D está correta porque a prática de "commits atômicos" preconiza o registro de mudanças pequenas e coerentes (unidades lógicas). Executar adds e commits separados para documentação e correção reflete a estrutura ideal recomendada pelos slides. As outras estão erradas porque realizar um "commit gigante" (adicionar todos e colocar "atualiza coisas") contrasta fortemente com os padrões.

Questão 22 — Alternativa D A alternativa D está correta porque o comando `git pull` representa uma operação composta que baixa as referências (fetch) e em seguida integra essas alterações remotas à branch local, tipicamente via merge. As outras estão erradas porque envios são executados pelo `git push`, e o `pull` não apaga ou exclui recursos remotamente.

Questão 23 — Alternativa C A alternativa C está correta porque a definição de DAG (Grafo Acíclico Direcionado) nos slides atesta que cada commit referencia seu ancestral criando um histórico direcional cujas ramificações nunca voltam de forma cíclica ao ponto inicial. As outras alternativas inventam lógicas contrárias, visto que a vantagem do modelo não linear é exatamente permitir bifurcações eficientes e não ficar preso a uma única arquitetura distribuída na nuvem.

Questão 24 — Alternativa B A alternativa B está correta porque reflete a sequência didática essencial exigida pelo material: inicializar o repositório (`init`), observar com o mapa do projeto (`status`), curar os arquivos para o próximo snapshot (`add`), e registrar no histórico (`commit`). As outras estão erradas porque apresentam comandos ou metodologias destrutivas/desordenadas que não seguem o fluxo de transição dos arquivos: untracked para staged para committed.

Questão 25 — Alternativa A A alternativa A está correta porque `git revert` é o comando seguro indicado no material para anular os efeitos de alterações de um commit já compartilhado sem destruir o histórico para a equipe. As outras estão erradas porque `reset` reescreveria e apagaria a história, o que é proibido após envio público, e `restore` trabalha com arquivos na fase antes do commit.

Questão 26 — Alternativa C A alternativa C está correta pois os slides afirmam claramente que "A tree é quem resolve o mapeamento entre conteúdo e nome" mapeando a estrutura de diretório, nomes e hashes correspondentes. As outras estão erradas, já que Blob não salva o nome, Tag é apenas uma referência para commit, e Index/Head não são objetos armazenados com esse fim.

Questão 27 — Alternativa B A alternativa B está correta porque o comando `git restore <arquivo>` reverte as mudanças ocorridas no diretório de trabalho de volta ao estado do último commit, apagando as modificações não desejadas que não entraram na Staging

Area. As outras alternativas estão erradas porque usam modificadores incorretos como o `--staged` (que removeria apenas da área de preparo) ou comandos não adequados ao erro relatado (`reset` opera após o commit ter sido efetuado).

Questão 28 — Alternativa E A alternativa E está correta porque a chave de leitura da documentação aponta que `[]` (colchetes) denotam argumentos opcionais para o comando requisitado. As outras estão incorretas porque opções mutuamente exclusivas e variáveis obrigatórias são expressas por sinais distintos na documentação do Git, como as chaves angulares `< >`.

Questão 29 — Alternativa B A alternativa B está correta porque a ética do versionamento no trabalho colaborativo requer que não se commite diretamente na branch principal (`main`), servindo ela como uma linha estável de trabalho integrado. As outras divergem abertamente da conduta esperada e ensinada no material para equipes.

Questão 30 — Alternativa C A alternativa C está correta porque um arquivo aparecerá como "staged" e "modified" se for adicionado à Staging Area (preparando a versão X) e depois editado no diretório de trabalho novamente antes de executar o commit (gerando a versão Y). As outras alternativas são errôneas pois tratam de erros e conflitos inventados, quando o sistema apenas segue sua natureza de capturar estados/snapshots